EXP12

September 23, 2026

[1]:
```
import kagglehub
path = kagglehub.dataset_download("zalando-research/fashionmnist")
```

```
Using Colab cache for faster access to the 'fashionmnist' dataset.
```

[2]:
```
import pandas as pd
import numpy as np
import os

# Locate the CSV files in the downloaded directory
csv_files = []
for root, dirs, files in os.walk(path):
    for file in files:
        if file.endswith('.csv'):
            csv_files.append(os.path.join(root, file))

print("Found CSV files:", csv_files)

# Fallback to keras datasets if no CSV is found, else load CSVs
if len(csv_files) >= 2:
    train_path = [f for f in csv_files if 'train' in f.lower()][0]
    test_path = [f for f in csv_files if 'test' in f.lower()][0]

    train_df = pd.read_csv(train_path)
    test_df = pd.read_csv(test_path)

    print("\n--- Train Dataset Info ---")
    print(train_df.info())
    print("\n--- Train Dataset Statistical Summary ---")
    print(train_df.describe())

    y_train_raw = train_df.iloc[:, 0].values
    x_train_raw = train_df.iloc[:, 1:].values
    y_test_raw = test_df.iloc[:, 0].values
    x_test_raw = test_df.iloc[:, 1:].values
else:
    print("No CSVs found in path. Loading from Keras as fallback with␣
 ↪equivalent structure.")
```

```
    from tensorflow.keras.datasets import fashion_mnist
    (x_train_3d, y_train_raw), (x_test_3d, y_test_raw) = fashion_mnist.
↪load_data()
    x_train_raw = x_train_3d.reshape(-1, 784)
    x_test_raw = x_test_3d.reshape(-1, 784)
```

```
Found CSV files: ['/kaggle/input/fashionmnist/fashion-mnist_test.csv',
'/kaggle/input/fashionmnist/fashion-mnist_train.csv']

--- Train Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 60000 entries, 0 to 59999
Columns: 785 entries, label to pixel784
dtypes: int64(785)
memory usage: 359.3 MB
None

--- Train Dataset Statistical Summary ---
              label        pixel1        pixel2        pixel3        pixel4  \
count  60000.000000  60000.000000  60000.000000  60000.000000  60000.000000
mean       4.500000      0.000900      0.006150      0.035333      0.101933
std        2.872305      0.094689      0.271011      1.222324      2.452871
min        0.000000      0.000000      0.000000      0.000000      0.000000
25%        2.000000      0.000000      0.000000      0.000000      0.000000
50%        4.500000      0.000000      0.000000      0.000000      0.000000
75%        7.000000      0.000000      0.000000      0.000000      0.000000
max        9.000000     16.000000     36.000000    226.000000    164.000000

             pixel5        pixel6        pixel7        pixel8        pixel9  \
count  60000.000000  60000.000000  60000.000000  60000.000000  60000.000000
mean       0.247967      0.411467      0.805767      2.198283      5.682000
std        4.306912      5.836188      8.215169     14.093378     23.819481
min        0.000000      0.000000      0.000000      0.000000      0.000000
25%        0.000000      0.000000      0.000000      0.000000      0.000000
50%        0.000000      0.000000      0.000000      0.000000      0.000000
75%        0.000000      0.000000      0.000000      0.000000      0.000000
max      227.000000    230.000000    224.000000    255.000000    254.000000

       …      pixel775      pixel776      pixel777      pixel778  \
count  …  60000.000000  60000.000000  60000.000000  60000.000000
mean   …     34.625400     23.300683     16.588267     17.869433
std    …     57.545242     48.854427     41.979611     43.966032
min    …      0.000000      0.000000      0.000000      0.000000
25%    …      0.000000      0.000000      0.000000      0.000000
50%    …      0.000000      0.000000      0.000000      0.000000
75%    …     58.000000      9.000000      0.000000      0.000000
max    …    255.000000    255.000000    255.000000    255.000000
```

```
           pixel779      pixel780      pixel781      pixel782      pixel783  \
count  60000.000000  60000.000000  60000.000000  60000.000000  60000.000000
mean      22.814817     17.911483      8.520633      2.753300      0.855517
std       51.830477     45.149388     29.614859     17.397652      9.356960
min        0.000000      0.000000      0.000000      0.000000      0.000000
25%        0.000000      0.000000      0.000000      0.000000      0.000000
50%        0.000000      0.000000      0.000000      0.000000      0.000000
75%        0.000000      0.000000      0.000000      0.000000      0.000000
max      255.000000    255.000000    255.000000    255.000000    255.000000

           pixel784
count  60000.00000
mean       0.07025
std        2.12587
min        0.00000
25%        0.00000
50%        0.00000
75%        0.00000
max      170.00000

[8 rows x 785 columns]
```

[3]:
```python
from tensorflow.keras.utils import to_categorical

# Normalize features
X_train = x_train_raw / 255.0
X_test = x_test_raw / 255.0

# One-hot encode targets
Y_train = to_categorical(y_train_raw, 10)
Y_test = to_categorical(y_test_raw, 10)

print(f"X_train shape: {X_train.shape}, Y_train shape: {Y_train.shape}")
print(f"X_test shape: {X_test.shape}, Y_test shape: {Y_test.shape}")
```

```
X_train shape: (60000, 784), Y_train shape: (60000, 10)
X_test shape: (10000, 784), Y_test shape: (10000, 10)
```

[4]:
```python
import time
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

results = {}

# Define activations to compare
activations = ['sigmoid', 'relu', 'tanh']
```

```
for act in activations:
    print(f"\nTraining network with {act.upper()} activation...")

    # Design the network
    model = Sequential([
        Dense(128, activation=act, input_shape=(784,)),
        Dense(64, activation=act),
        Dense(10, activation='softmax')
    ])

    # Compile
    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    # Train & record time
    start_time = time.time()
    history = model.fit(X_train, Y_train, epochs=8, batch_size=64,␣
 ↪validation_split=0.1, verbose=1)
    end_time = time.time()

    training_time = end_time - start_time

    # Evaluate
    loss, accuracy = model.evaluate(X_test, Y_test, verbose=0)

    results[act] = {
        'history': history.history,
        'training_time': training_time,
        'accuracy': accuracy
    }
    print(f"{act.upper()} - Training Time: {training_time:.2f}s, Test Accuracy:␣
 ↪{accuracy:.4f}")
```

```
Training network with SIGMOID activation…

/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Epoch 1/8
844/844             5s 4ms/step -
accuracy: 0.7661 - loss: 0.7343 - val_accuracy: 0.8267 - val_loss: 0.4859
Epoch 2/8
844/844             5s 6ms/step -
accuracy: 0.8473 - loss: 0.4260 - val_accuracy: 0.8508 - val_loss: 0.4209
```

```
Epoch 3/8
844/844             4s 4ms/step -
accuracy: 0.8643 - loss: 0.3782 - val_accuracy: 0.8612 - val_loss: 0.3839
Epoch 4/8
844/844             3s 4ms/step -
accuracy: 0.8732 - loss: 0.3511 - val_accuracy: 0.8685 - val_loss: 0.3635
Epoch 5/8
844/844             4s 5ms/step -
accuracy: 0.8793 - loss: 0.3302 - val_accuracy: 0.8713 - val_loss: 0.3604
Epoch 6/8
844/844             4s 4ms/step -
accuracy: 0.8846 - loss: 0.3178 - val_accuracy: 0.8742 - val_loss: 0.3501
Epoch 7/8
844/844             4s 4ms/step -
accuracy: 0.8890 - loss: 0.3049 - val_accuracy: 0.8778 - val_loss: 0.3380
Epoch 8/8
844/844             4s 4ms/step -
accuracy: 0.8919 - loss: 0.2940 - val_accuracy: 0.8847 - val_loss: 0.3259
SIGMOID - Training Time: 32.62s, Test Accuracy: 0.8806

Training network with RELU activation…
Epoch 1/8
844/844             5s 4ms/step -
accuracy: 0.8128 - loss: 0.5267 - val_accuracy: 0.8418 - val_loss: 0.4377
Epoch 2/8
844/844             3s 4ms/step -
accuracy: 0.8596 - loss: 0.3872 - val_accuracy: 0.8538 - val_loss: 0.3983
Epoch 3/8
844/844             4s 5ms/step -
accuracy: 0.8747 - loss: 0.3460 - val_accuracy: 0.8567 - val_loss: 0.3919
Epoch 4/8
844/844             4s 5ms/step -
accuracy: 0.8832 - loss: 0.3201 - val_accuracy: 0.8702 - val_loss: 0.3577
Epoch 5/8
844/844             4s 4ms/step -
accuracy: 0.8887 - loss: 0.3038 - val_accuracy: 0.8742 - val_loss: 0.3484
Epoch 6/8
844/844             4s 5ms/step -
accuracy: 0.8926 - loss: 0.2889 - val_accuracy: 0.8707 - val_loss: 0.3594
Epoch 7/8
844/844             5s 5ms/step -
accuracy: 0.8985 - loss: 0.2741 - val_accuracy: 0.8775 - val_loss: 0.3402
Epoch 8/8
844/844             3s 4ms/step -
accuracy: 0.9009 - loss: 0.2645 - val_accuracy: 0.8845 - val_loss: 0.3298
RELU - Training Time: 32.19s, Test Accuracy: 0.8860

Training network with TANH activation…
```

```
Epoch 1/8
844/844             6s 7ms/step -
accuracy: 0.8244 - loss: 0.4963 - val_accuracy: 0.8467 - val_loss: 0.4299
Epoch 2/8
844/844             4s 4ms/step -
accuracy: 0.8633 - loss: 0.3760 - val_accuracy: 0.8613 - val_loss: 0.3827
Epoch 3/8
844/844             3s 4ms/step -
accuracy: 0.8757 - loss: 0.3376 - val_accuracy: 0.8695 - val_loss: 0.3594
Epoch 4/8
844/844             7s 6ms/step -
accuracy: 0.8832 - loss: 0.3194 - val_accuracy: 0.8773 - val_loss: 0.3401
Epoch 5/8
844/844             3s 4ms/step -
accuracy: 0.8896 - loss: 0.2993 - val_accuracy: 0.8712 - val_loss: 0.3466
Epoch 6/8
844/844             3s 4ms/step -
accuracy: 0.8939 - loss: 0.2868 - val_accuracy: 0.8758 - val_loss: 0.3348
Epoch 7/8
844/844             4s 4ms/step -
accuracy: 0.9002 - loss: 0.2716 - val_accuracy: 0.8752 - val_loss: 0.3435
Epoch 8/8
844/844             4s 5ms/step -
accuracy: 0.9033 - loss: 0.2611 - val_accuracy: 0.8852 - val_loss: 0.3103
TANH - Training Time: 35.16s, Test Accuracy: 0.8868
```

[5]:
```python
import matplotlib.pyplot as plt

# Create comparison DataFrame
comparison_df = pd.DataFrame({
    'Activation Function': [act.upper() for act in results.keys()],
    'Training Time (seconds)': [results[act]['training_time'] for act in␣
 ↪results.keys()],
    'Classification Accuracy': [results[act]['accuracy'] for act in results.
 ↪keys()]
})

display(comparison_df)

# Plotting Classification Accuracy Comparison
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.bar(comparison_df['Activation Function'], comparison_df['Classification␣
 ↪Accuracy'], color=['blue', 'orange', 'green'])
plt.title('Classification Accuracy Comparison')
plt.xlabel('Activation Function')
```

```
plt.ylabel('Accuracy')
plt.ylim(0, 1.0)
for i, val in enumerate(comparison_df['Classification Accuracy']):
    plt.text(i, val + 0.02, f"{val:.4f}", ha='center')

# Plotting Training Time Comparison
plt.subplot(1, 2, 2)
plt.bar(comparison_df['Activation Function'], comparison_df['Training Time␣
 ↪(seconds)'], color=['blue', 'orange', 'green'])
plt.title('Training Time Comparison')
plt.xlabel('Activation Function')
plt.ylabel('Training Time (s)')
for i, val in enumerate(comparison_df['Training Time (seconds)']):
    plt.text(i, val + 0.1, f"{val:.2f}s", ha='center')

plt.tight_layout()
plt.show()
```

	Activation Function	Training Time (seconds)	Classification Accuracy
0	SIGMOID	32.615285	0.8806
1	RELU	32.185057	0.8860
2	TANH	35.155779	0.8868

